

sheet_2_student_handout_m_Q3

May 16, 2026

1 Computer Animation - Assignment Sheet 2

```
[1]: import numpy as np
from typing import Optional, Union
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from mpl_toolkits.mplot3d.art3d import Poly3DCollection

try:
    from IPython.display import HTML, display
except ImportError:
    HTML = None
    display = None

np.set_printoptions(precision=5, suppress=True)
EPS = 1e-10
```

```
[2]: #for visuals
try:
    shell = get_ipython()
    try:
        shell.run_line_magic("matplotlib", "widget")
        print("Interactive backend enabled: %matplotlib widget")
    except Exception:
        shell.run_line_magic("matplotlib", "notebook")
        print("Interactive backend enabled: %matplotlib notebook")
except Exception:
    print("Interactive backend not enabled. For draggable 3D plots in JupyterLab, install ipympl and run `%matplotlib widget`.")
```

Interactive backend enabled: %matplotlib widget

1.1 Provided helper functions from Sheet 1

```
[3]: # -----
# Conventions used in this notebook
# -----
# Quaternions are represented as [w, x, y, z].
```

```

# Euler angles are represented in the order specified by `order`.
# Example: order="ZYX" means euler=[z_angle, y_angle, x_angle], and
# R = Rz(z_angle) @ Ry(y_angle) @ Rx(x_angle).
# All six Tait-Bryan Euler orders are supported:
# "XYZ", "XZY", "YXZ", "YZX", "ZXY", "ZYX".
# -----

ORDER = "ZYX"

def normalize_vector(v: np.ndarray, axis: int = -1, eps: float = EPS) -> np.
    ndarray:
    """Normalize vectors along the given axis."""
    v = np.asarray(v, dtype=float)
    n = np.linalg.norm(v, axis=axis, keepdims=True)
    if np.any(n < eps):
        raise ValueError("Cannot normalize a vector with norm close to zero.")
    return v / n

def normalize_quaternion(q: np.ndarray, eps: float = EPS) -> np.ndarray:
    """Normalize one quaternion of shape (4,) or an array of quaternions (...
    ↪4)."""
    q = np.asarray(q, dtype=float)
    if q.shape[-1] != 4:
        raise ValueError("Quaternions must have shape (... , 4).")
    n = np.linalg.norm(q, axis=-1, keepdims=True)
    if np.any(n < eps):
        raise ValueError("Cannot normalize a quaternion with norm close to zero.
    ↪")
    return q / n

def quaternion_conjugate(q: np.ndarray) -> np.ndarray:
    """Return the conjugate of q = [w, x, y, z]."""
    q = np.asarray(q, dtype=float)
    if q.shape[-1] != 4:
        raise ValueError("Quaternions must have shape (... , 4).")
    out = q.copy()
    out[..., 1:] *= -1.0
    return out

def quaternion_norm(q: np.ndarray) -> np.ndarray:
    """Return the Euclidean norm of one quaternion or a batch of quaternions."""
    q = np.asarray(q, dtype=float)
    if q.shape[-1] != 4:
        raise ValueError("Quaternions must have shape (... , 4).")

```

```

return np.linalg.norm(q, axis=-1)

def quaternion_multiply(q_left: np.ndarray, q_right: np.ndarray) -> np.ndarray:
    """Hamilton product q_left * q_right for quaternions [w, x, y, z]."""
    q_left = np.asarray(q_left, dtype=float)
    q_right = np.asarray(q_right, dtype=float)
    if q_left.shape[-1] != 4 or q_right.shape[-1] != 4:
        raise ValueError("Both inputs must have shape (... , 4).")

    w1, x1, y1, z1 = np.moveaxis(q_left, -1, 0)
    w2, x2, y2, z2 = np.moveaxis(q_right, -1, 0)

    return np.stack([
        w1*w2 - x1*x2 - y1*y2 - z1*z2,
        w1*x2 + x1*w2 + y1*z2 - z1*y2,
        w1*y2 - x1*z2 + y1*w2 + z1*x2,
        w1*z2 + x1*y2 - y1*x2 + z1*w2,
    ], axis=-1)

def rotation_matrix_x(angle: float) -> np.ndarray:
    """Rotation matrix for a rotation about the x-axis."""
    c, s = np.cos(angle), np.sin(angle)
    return np.array([[1.0, 0.0, 0.0], [0.0, c, -s], [0.0, s, c]])

def rotation_matrix_y(angle: float) -> np.ndarray:
    """Rotation matrix for a rotation about the y-axis."""
    c, s = np.cos(angle), np.sin(angle)
    return np.array([[c, 0.0, s], [0.0, 1.0, 0.0], [-s, 0.0, c]])

def rotation_matrix_z(angle: float) -> np.ndarray:
    """Rotation matrix for a rotation about the z-axis."""
    c, s = np.cos(angle), np.sin(angle)
    return np.array([[c, -s, 0.0], [s, c, 0.0], [0.0, 0.0, 1.0]])

def axis_rotation_matrix(axis: str, angle: float) -> np.ndarray:
    """Return a basic rotation matrix for the selected axis."""
    axis = axis.upper()
    if axis == "X":
        return rotation_matrix_x(angle)
    if axis == "Y":
        return rotation_matrix_y(angle)
    if axis == "Z":

```

```

        return rotation_matrix_z(angle)
    raise ValueError("axis must be 'X', 'Y', or 'Z'.")

def axis_angle_quaternion(axis: str, angle: float) -> np.ndarray:
    """Quaternion [w, x, y, z] for a rotation around one coordinate axis."""
    half = angle / 2.0
    c, s = np.cos(half), np.sin(half)
    axis = axis.upper()
    if axis == "X":
        return np.array([c, s, 0.0, 0.0])
    if axis == "Y":
        return np.array([c, 0.0, s, 0.0])
    if axis == "Z":
        return np.array([c, 0.0, 0.0, s])
    raise ValueError("axis must be 'X', 'Y', or 'Z'.")

def euler_to_rotation_matrix(euler: np.ndarray, order: str = ORDER) -> np.
    ndarray:
    """
    Convert Euler angles to a rotation matrix.

    The angles are interpreted in the same order as the order string.
    Example: order="ZYX", euler=[z_angle, y_angle, x_angle].
    """
    euler = np.asarray(euler, dtype=float).reshape(3,)

    R = np.eye(3)
    for axis, angle in zip(order, euler):
        R = R @ axis_rotation_matrix(axis, angle)
    return R

def euler_to_quaternion(euler: np.ndarray, order: str = ORDER) -> np.ndarray:
    """
    Convert Euler angles to a unit quaternion [w, x, y, z].

    The angles are interpreted in the same order as the order string.
    Example: order="ZYX", euler=[z_angle, y_angle, x_angle].
    """
    euler = np.asarray(euler, dtype=float).reshape(3,)

    q = np.array([1.0, 0.0, 0.0, 0.0])
    for axis, angle in zip(order, euler):
        q = quaternion_multiply(q, axis_angle_quaternion(axis, angle))
    return normalize_quaternion(q)

```

```

def quaternion_to_rotation_matrix(q: np.ndarray) -> np.ndarray:
    """Convert  $q = [w, x, y, z]$  to a  $3 \times 3$  rotation matrix."""
    q = normalize_quaternion(np.asarray(q, dtype=float).reshape(4,))
    w, x, y, z = q
    return np.array([
        [1 - 2*(y*y + z*z), 2*(x*y - w*z), 2*(x*z + w*y)],
        [2*(x*y + w*z), 1 - 2*(x*x + z*z), 2*(y*z - w*x)],
        [2*(x*z - w*y), 2*(y*z + w*x), 1 - 2*(x*x + y*y)],
    ], dtype=float)

def rotation_matrix_to_quaternion(R: np.ndarray) -> np.ndarray:
    """Convert a  $3 \times 3$  rotation matrix to a unit quaternion  $[w, x, y, z]$ ."""
    R = np.asarray(R, dtype=float).reshape(3, 3)
    trace = np.trace(R)

    if trace > 0.0:
        s = 2.0 * np.sqrt(trace + 1.0)
        w = 0.25 * s
        x = (R[2, 1] - R[1, 2]) / s
        y = (R[0, 2] - R[2, 0]) / s
        z = (R[1, 0] - R[0, 1]) / s
    elif R[0, 0] > R[1, 1] and R[0, 0] > R[2, 2]:
        s = 2.0 * np.sqrt(1.0 + R[0, 0] - R[1, 1] - R[2, 2])
        w = (R[2, 1] - R[1, 2]) / s
        x = 0.25 * s
        y = (R[0, 1] + R[1, 0]) / s
        z = (R[0, 2] + R[2, 0]) / s
    elif R[1, 1] > R[2, 2]:
        s = 2.0 * np.sqrt(1.0 + R[1, 1] - R[0, 0] - R[2, 2])
        w = (R[0, 2] - R[2, 0]) / s
        x = (R[0, 1] + R[1, 0]) / s
        y = 0.25 * s
        z = (R[1, 2] + R[2, 1]) / s
    else:
        s = 2.0 * np.sqrt(1.0 + R[2, 2] - R[0, 0] - R[1, 1])
        w = (R[1, 0] - R[0, 1]) / s
        x = (R[0, 2] + R[2, 0]) / s
        y = (R[1, 2] + R[2, 1]) / s
        z = 0.25 * s

    return normalize_quaternion(np.array([w, x, y, z]))

```

```

def rotation_matrix_to_euler(R: np.ndarray, order: str = ORDER, eps: float = 1e-8) -> np.ndarray:
    """
    Convert a rotation matrix to Euler angles for any of the six supported
    orders.

    Returns angles in the same order as `order`.
    Example: order="ZYX" returns [z_angle, y_angle, x_angle].
    In gimbal-lock cases, the last angle is set to zero.
    """
    R = np.asarray(R, dtype=float).reshape(3, 3)

    if order == "XYZ":
        sb = np.clip(R[0, 2], -1.0, 1.0)
        b = np.arcsin(sb)
        if abs(abs(sb) - 1.0) < eps:
            c = 0.0
            a = np.arctan2(R[1, 0], R[1, 1]) if sb > 0 else np.arctan2(-R[1, 0], R[1, 1])
        else:
            a = np.arctan2(-R[1, 2], R[2, 2])
            c = np.arctan2(-R[0, 1], R[0, 0])

    elif order == "XZY":
        sb = np.clip(-R[0, 1], -1.0, 1.0)
        b = np.arcsin(sb)
        if abs(abs(sb) - 1.0) < eps:
            c = 0.0
            a = np.arctan2(-R[1, 2], R[1, 0]) if sb > 0 else np.arctan2(-R[1, 2], -R[1, 0])
        else:
            a = np.arctan2(R[2, 1], R[1, 1])
            c = np.arctan2(R[0, 2], R[0, 0])

    elif order == "YXZ":
        sb = np.clip(-R[1, 2], -1.0, 1.0)
        b = np.arcsin(sb)
        if abs(abs(sb) - 1.0) < eps:
            c = 0.0
            a = np.arctan2(-R[2, 0], R[0, 0])
        else:
            a = np.arctan2(R[0, 2], R[2, 2])
            c = np.arctan2(R[1, 0], R[1, 1])

    elif order == "YZX":
        sb = np.clip(R[1, 0], -1.0, 1.0)
        b = np.arcsin(sb)

```

```

        if abs(abs(sb) - 1.0) < eps:
            c = 0.0
            a = np.arctan2(R[0, 2], R[2, 2])
        else:
            a = np.arctan2(-R[2, 0], R[0, 0])
            c = np.arctan2(-R[1, 2], R[1, 1])

    elif order == "ZXY":
        sb = np.clip(R[2, 1], -1.0, 1.0)
        b = np.arcsin(sb)
        if abs(abs(sb) - 1.0) < eps:
            c = 0.0
            a = np.arctan2(R[1, 0], R[0, 0])
        else:
            a = np.arctan2(-R[0, 1], R[1, 1])
            c = np.arctan2(-R[2, 0], R[2, 2])

    elif order == "ZYX":
        sb = np.clip(-R[2, 0], -1.0, 1.0)
        b = np.arcsin(sb)
        if abs(abs(sb) - 1.0) < eps:
            c = 0.0
            a = np.arctan2(R[1, 2], R[0, 2]) if sb > 0 else np.arctan2(-R[1, 2], -R[0, 2])
        else:
            a = np.arctan2(R[1, 0], R[0, 0])
            c = np.arctan2(R[2, 1], R[2, 2])

    return np.array([a, b, c])

def quaternion_to_euler(q: np.ndarray, order: str = ORDER) -> np.ndarray:
    """Convert a quaternion [w, x, y, z] to Euler angles in the requested order.
    """
    return rotation_matrix_to_euler(quaternion_to_rotation_matrix(q),
    order=order)

def rotation_direction_from_matrix(R: np.ndarray) -> np.ndarray:
    """Direction of the local +x axis after applying R."""
    return R @ np.array([1.0, 0.0, 0.0])

def directions_from_quaternions(quaternions: np.ndarray) -> np.ndarray:
    """Map a sequence of quaternions to the corresponding local +x directions.
    """

```

```

    return np.
    ↪array([rotation_direction_from_matrix(quaternion_to_rotation_matrix(q)) for
    ↪q in quaternions])

def directions_from_eulers(eulers: np.ndarray, order: str = ORDER) -> np.
    ↪ndarray:
    """Map a sequence of Euler angles to the corresponding local +x directions.
    ↪"""
    return np.array([rotation_direction_from_matrix(euler_to_rotation_matrix(e,
    ↪order=order)) for e in eulers])

def relative_rotation_angle(R0: np.ndarray, R1: np.ndarray) -> float:
    """Return the angle of the relative rotation R0.T @ R1 in radians."""
    R_rel = R0.T @ R1
    c = (np.trace(R_rel) - 1.0) / 2.0
    return float(np.arccos(np.clip(c, -1.0, 1.0)))

```

1.2 Provided mesh and plotting helpers for Assignment 3

```

[4]: ARROW_LENGTH = 0.45
    SPHERE_RADIUS = 1.0

def create_sphere_mesh(radius: float = SPHERE_RADIUS, n_lat: int = 24, n_lon:
    ↪int = 48):
    """Return x, y, z arrays for plotting a sphere with Matplotlib."""
    u = np.linspace(0.0, 2.0*np.pi, n_lon)
    v = np.linspace(0.0, np.pi, n_lat)
    x = radius * np.outer(np.cos(u), np.sin(v))
    y = radius * np.outer(np.sin(u), np.sin(v))
    z = radius * np.outer(np.ones_like(u), np.cos(v))
    return x, y, z

def create_arrow_mesh(length: float = ARROW_LENGTH,
    shaft_length: float = 0.28,
    shaft_radius: float = 0.025,
    head_radius: float = 0.085,
    n_segments: int = 16):
    """
    Create a simple arrow mesh whose tip points along the local negative x-axis.

    Returns
    -----
    vertices : ndarray, shape (n_vertices, 3)
    """

```



```

faces : list[list[int]]
    Indices into vertices. Faces can be triangles or quads.
    """
    theta = np.linspace(0.0, 2.0*np.pi, n_segments, endpoint=False)

    base = np.column_stack([
        np.zeros(n_segments),
        shaft_radius * np.cos(theta),
        shaft_radius * np.sin(theta),
    ])
    shaft_end = np.column_stack([
        -np.full(n_segments, shaft_length),
        shaft_radius * np.cos(theta),
        shaft_radius * np.sin(theta),
    ])
    head_base = np.column_stack([
        -np.full(n_segments, shaft_length),
        head_radius * np.cos(theta),
        head_radius * np.sin(theta),
    ])
    tip = np.array([-length, 0.0, 0.0])
    base_center = np.array([0.0, 0.0, 0.0])

    vertices = np.vstack([base, shaft_end, head_base, tip, base_center])
    tip_idx = 3 * n_segments
    base_center_idx = 3 * n_segments + 1

    faces = []
    for i in range(n_segments):
        j = (i + 1) % n_segments
        faces.append([i, j, n_segments + j, n_segments + i])
        faces.append([n_segments + i, n_segments + j, 2*n_segments + j,
↪2*n_segments + i])
        faces.append([2*n_segments + i, 2*n_segments + j, tip_idx])
        faces.append([base_center_idx, j, i])

    return vertices, faces

def transform_vertices(vertices: np.ndarray,
                       R: np.ndarray,
                       offset: Optional[np.ndarray] = None,
                       scale: float = 1.0) -> np.ndarray:
    """Apply scale, rotation, and translation to vertices."""
    vertices = np.asarray(vertices, dtype=float)
    R = np.asarray(R, dtype=float).reshape(3, 3)
    if offset is None:

```

```

        offset = np.zeros(3)
    offset = np.asarray(offset, dtype=float).reshape(3,)
    return scale * (vertices @ R.T) + offset

def add_arrow(ax,
              vertices: np.ndarray,
              faces: list,
              R: np.ndarray,
              offset: np.ndarray,
              color: str = "tab:blue",
              alpha: float = 0.55):
    """Add one transformed arrow mesh to a 3D axis."""
    transformed = transform_vertices(vertices, R, offset=offset)
    poly = [[transformed[idx] for idx in face] for face in faces]
    collection = Poly3DCollection(poly, alpha=alpha, linewidths=0.4)
    collection.set_facecolor(color)
    collection.set_edgecolor("black")
    ax.add_collection3d(collection)
    return collection

def inward_arrow_offset(direction: np.ndarray,
                       sphere_radius: float = SPHERE_RADIUS,
                       arrow_length: float = ARROW_LENGTH) -> np.ndarray:
    """
    Place the arrow base outside the sphere so the arrow tip lies on the sphere.
    """
    direction = normalize_vector(np.asarray(direction, dtype=float).reshape(3,))
    return (sphere_radius + arrow_length) * direction

def set_axes_equal(ax, limit: float = 1.75):
    """Use equal limits on all three 3D axes."""
    ax.set_xlim(-limit, limit)
    ax.set_ylim(-limit, limit)
    ax.set_zlim(-limit, limit)
    ax.set_box_aspect((1, 1, 1))
    ax.set_xlabel("x")
    ax.set_ylabel("y")
    ax.set_zlabel("z")

def plot_interpolation_paths(q_sequence: np.ndarray,
                             euler_sequence: np.ndarray,
                             order: str = ORDER,
                             sample_indices: Optional[np.ndarray] = None,

```

```

        title: str = "Quaternion SLERP vs. Euler LERP"):
    """Plot the orientation paths of the local +x axis on the sphere."""
    q_sequence = np.asarray(q_sequence, dtype=float)
    euler_sequence = np.asarray(euler_sequence, dtype=float)

    q_dirs = directions_from_quaternions(q_sequence)
    e_dirs = directions_from_eulers(euler_sequence, order=order)

    vertices, faces = create_arrow_mesh()
    sphere_x, sphere_y, sphere_z = create_sphere_mesh()

    if sample_indices is None:
        sample_indices = np.linspace(0, len(q_sequence) - 1, 7, dtype=int)

    fig = plt.figure(figsize=(8, 7))
    ax = fig.add_subplot(111, projection="3d")
    ax.plot_wireframe(sphere_x, sphere_y, sphere_z, linewidth=0.25, alpha=0.25)
    ax.plot(q_dirs[:, 0], q_dirs[:, 1], q_dirs[:, 2], color="tab:blue",
    ↪linewidth=2.0, label="quaternion SLERP")
    ax.plot(e_dirs[:, 0], e_dirs[:, 1], e_dirs[:, 2], color="tab:green",
    ↪linewidth=2.0, label="Euler LERP")
    ax.scatter(q_dirs[[0, -1], 0], q_dirs[[0, -1], 1], q_dirs[[0, -1], 2],
    ↪color="black", s=35)

    for idx in sample_indices:
        q_R = quaternion_to_rotation_matrix(q_sequence[idx])
        e_R = euler_to_rotation_matrix(euler_sequence[idx], order=order)
        add_arrow(ax, vertices, faces, q_R,
    ↪offset=inward_arrow_offset(q_dirs[idx]), color="tab:blue", alpha=0.35)
        add_arrow(ax, vertices, faces, e_R,
    ↪offset=inward_arrow_offset(e_dirs[idx]), color="tab:green", alpha=0.35)

    set_axes_equal(ax)
    ax.set_title(title + f" (Euler order: {order})")
    ax.legend(loc="upper left")
    return fig, ax

def plot_angular_distance(t_values: np.ndarray,
                          q_sequence: np.ndarray,
                          euler_sequence: np.ndarray,
                          euler_start: np.ndarray,
                          order: str = ORDER):
    """Plot angular distance from the start orientation over interpolation time.
    ↪"""
    t_values = np.asarray(t_values, dtype=float)

```

```

R0 = euler_to_rotation_matrix(euler_start, order=order)

q_angles = np.array([
    relative_rotation_angle(R0, quaternion_to_rotation_matrix(q))
    for q in q_sequence
])
e_angles = np.array([
    relative_rotation_angle(R0, euler_to_rotation_matrix(e, order=order))
    for e in euler_sequence
])

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(t_values, np.rad2deg(q_angles), label="quaternion SLERP")
ax.plot(t_values, np.rad2deg(e_angles), label="Euler LERP")
ax.set_xlabel("t")
ax.set_ylabel("angular distance from start [degrees]")
ax.set_title("Interpolation progress over time")
ax.grid(True, alpha=0.3)
ax.legend()
return fig, ax

```

1.3 Assignment 3: Animation

```

[5]: def animate_interpolation(q_sequence: np.ndarray,
                                euler_sequence: np.ndarray,
                                order: str = ORDER,
                                interval: int = 70):

    q_dirs = directions_from_quaternions(q_sequence)
    e_dirs = directions_from_eulers(euler_sequence)

    vertices, faces = create_arrow_mesh()
    sphere_x, sphere_y, sphere_z = create_sphere_mesh()

    fig = plt.figure(figsize=(8, 7))
    ax = fig.add_subplot(111, projection="3d")

    # sphere wireframe- static
    ax.plot_wireframe(sphere_x, sphere_y, sphere_z, linewidth=0.25, alpha=0.25)

    ax.plot(q_dirs[:, 0], q_dirs[:, 1], q_dirs[:, 2],
            color="tab:blue", linewidth=1.5, alpha=0.35, label="quaternion ↪ SLERP")
    ax.plot(e_dirs[:, 0], e_dirs[:, 1], e_dirs[:, 2],
            color="tab:green", linewidth=1.5, alpha=0.35, label="Euler LERP")

    set_axes_equal(ax)

```

```

ax.legend(loc="upper left")
ax.set_title(f"Quaternion SLERP vs. Euler LERP (Euler order: {order})")

q_coll = [None]
e_coll = [None]

# dynamic part
def update(frame):
    if q_coll[0] is not None:
        q_coll[0].remove()
    if e_coll[0] is not None:
        e_coll[0].remove()

    q_R = quaternion_to_rotation_matrix(q_sequence[frame])
    e_R = euler_to_rotation_matrix(euler_sequence[frame], order=order)

    q_coll[0] = add_arrow(ax, vertices, faces, q_R,
                          offset=inward_arrow_offset(q_dirs[frame]),
                          color="tab:blue", alpha=0.8)
    e_coll[0] = add_arrow(ax, vertices, faces, e_R,
                          offset=inward_arrow_offset(e_dirs[frame]),
                          color="tab:green", alpha=0.8)

    return q_coll[0], e_coll[0]

return FuncAnimation(fig, update, frames=len(q_sequence),
                    interval=interval, blit=False)

# raise NotImplementedError("TODO")

```

1.4 Assignment 2.1: Spherical linear interpolation of quaternions

```

[6]: def slerp_quaternion(q_start: np.ndarray, q_end: np.ndarray, t: Union[np.
    ndarray, float]) -> np.ndarray:
    """
    Note:
    Spherical Linear Interpolation (SLERP) between two quaternions.
    performs smooth interpolation along the shortest path on the unit
    quaternion sphere.

    Args:
    q_start: Starting quaternion [w, x, y, z]
    q_end: Ending quaternion [w, x, y, z]
    t: Interpolation parameter(s), scalar or 1D array with values in [0, 1]
    - interpolation factors? 0 = q_start, 1 = q_end

    Returns:
    """

```

```

Interpolated quaternions with same shape as t (scalar or array)
"""
# inputs are numpy arrays and normalize
q_start = normalize_quaternion(np.asarray(q_start, dtype=float).reshape(4,))
q_end = normalize_quaternion(np.asarray(q_end, dtype=float).reshape(4,))

# Handle sign ambiguity: shortest path by checking dot product
# If dot product is negative, the quaternions point in opposite directions
# Flipping q_end to ensure we take the shortest path

dot_product = np.dot(q_start, q_end)
if dot_product < 0.0:
    q_end = -q_end
    dot_product = -dot_product

# Clamp dot product to avoid numerical errors in arccos
dot_product = np.clip(dot_product, -1.0, 1.0)

# Calculate the angle between quaternions
theta = np.arccos(dot_product)

# Convert t to numpy array for vectorized operations
t = np.atleast_1d(np.asarray(t, dtype=float))
is_scalar = t.shape == (1,)

# Handle the case where quaternions are very close theta == 0
# Use linear interpolation to avoid division by zero
sin_theta = np.sin(theta)

if abs(sin_theta) < 1e-6:
    # Quaternions are nearly identical, use linear interpolation
    weights_start = 1.0 - t
    weights_end = t
else:
    # Standard SLERP formula:
    #  $q(t) = [\sin((1-t)\theta) / \sin(\theta)] * q\_start + [\sin(t\theta) / \sin(\theta)] * q\_end$ 

    weights_start = np.sin((1.0 - t) * theta) / sin_theta
    weights_end = np.sin(t * theta) / sin_theta

# Perform interpolation
# Shape handling: weights can broadcast with quaternion
result = (weights_start[:, np.newaxis] * q_start +
          weights_end[:, np.newaxis] * q_end)

# Normalize result quaternions

```

```

result = normalize_quaternion(result)

# Return scalar if input was scalar
if is_scalar:
    return result[0]
return result

```

1.5 Assignment 2.2: Linear interpolation of Euler angles

```

[7]: def lerp_euler(euler_start: np.ndarray, euler_end: np.ndarray, t: Union[np.
    ndarray, float]) -> np.ndarray:
    """
    Linear Interpolation (LERP) of Euler angles.

    Performs simple component-wise linear interpolation of Euler angles.
    Note: This is a naive approach and may not produce the most intuitive
    ↪rotations
    near gimbal lock configurations, but it's straightforward and works well for
    most practical cases with small angle differences.

    Args:
        euler_start: Starting Euler angles [angle1, angle2, angle3] in radians
        euler_end: Ending Euler angles [angle1, angle2, angle3] in radians
        t: Interpolation parameter(s), scalar or 1D array with values in [0, 1]

    Returns:
        Interpolated Euler angle(s) in radians
    """
    euler_start = np.asarray(euler_start, dtype=float).reshape(3,)
    euler_end = np.asarray(euler_end, dtype=float).reshape(3,)

    # Convert t to numpy array for vectorized operations
    t = np.atleast_1d(np.asarray(t, dtype=float))
    is_scalar = t.shape == (1,)

    # Component-wise linear interpolation
    # result(t) = (1-t) * euler_start + t * euler_end
    weights_start = 1.0 - t
    weights_end = t

    # interpolation: broadcast weights to multiply with euler angles
    result = (weights_start[:, np.newaxis] * euler_start +
              weights_end[:, np.newaxis] * euler_end)

    if is_scalar:
        return result[0]
    return result

```

1.6 Assignment 3: Illustration and comparison

```
[8]: # Example setup for Assignment 3.

ORDER = "ZYX"
t_values = np.linspace(0.0, 1.0, 61)

euler_start = np.deg2rad([-95.0, -55.0, 10.0])
euler_end = np.deg2rad([135.0, 70.0, 165.0])

q_start = euler_to_quaternion(euler_start, order=ORDER)
q_end = euler_to_quaternion(euler_end, order=ORDER)

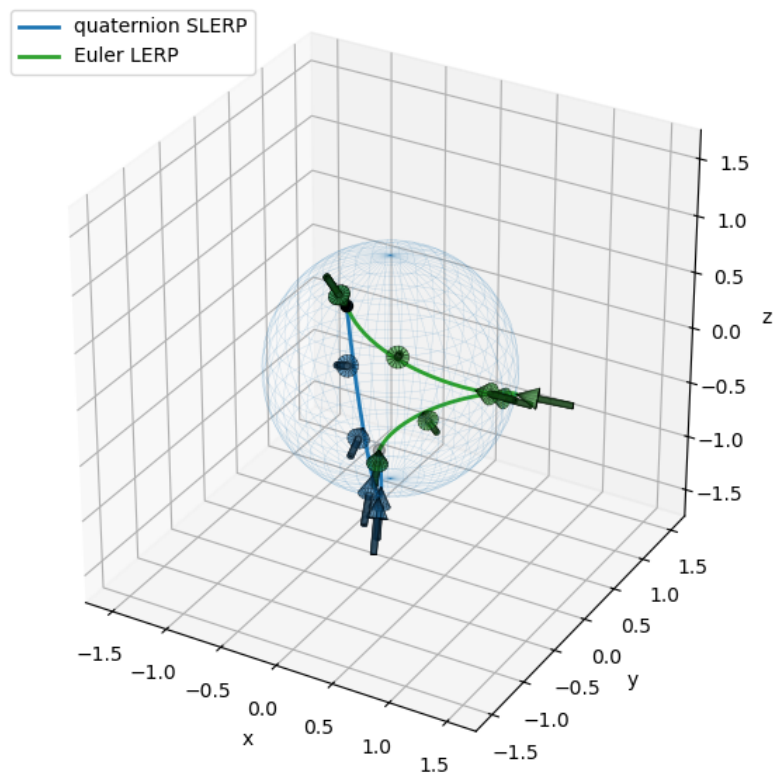
# TODO
q_sequence = slerp_quaternion(q_start, q_end, t_values)
euler_sequence = lerp_euler(euler_start, euler_end, t_values)

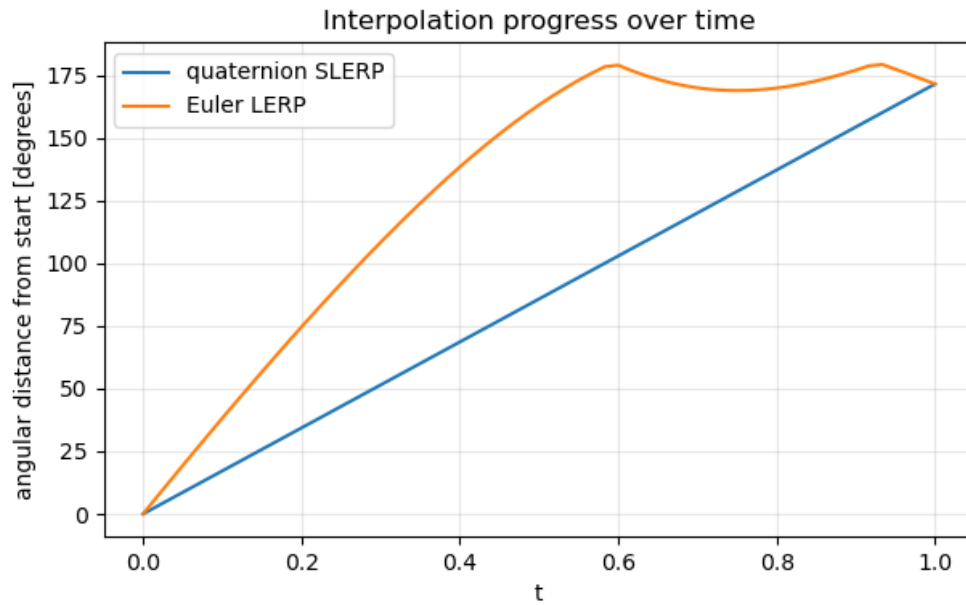
fig, ax = plot_interpolation_paths(q_sequence, euler_sequence, order=ORDER)
plt.show()

fig2, ax2 = plot_angular_distance(t_values, q_sequence, euler_sequence,
    ↪ euler_start, order=ORDER)
plt.show()

anim = animate_interpolation(q_sequence, euler_sequence, order=ORDER,
    ↪ interval=70)
if HTML is not None:
    display(HTML(anim.to_jshtml()))
```


Quaternion SLERP vs. Euler LERP (Euler order: ZYX)





<IPython.core.display.HTML object>

1.6.1 Observations:

The SLERP path traces a geodesic arc (the shortest path between two points on a sphere) — it's a smooth, consistent curve.

The Euler LERP path takes a different, longer route as it curves away from the geodesic because blending three angles independently doesn't correspond to a straight rotation path.

SLERP moves at a constant angular velocity, as the angular distance from the plot shows a straight, this is not the case with Euler LERP.